

1. A reproducible evaluation harness comparing both paradigms on all 300 SWE-bench Lite tasks [Ulfsnes et al.(2024)Ulfsnes, Moe, Stray, and Skarpen].
2. Formal graph-relevance scoring quantifying retrieval precision at $K \in \{1, 3, 5, 10\}$ [Ai et al.(2025)Ai, Zhan, and Liu].
3. An ablation study decomposing performance gains attributable to graph topology versus semantic embedding quality [Wang et al.(2023)Wang, Ding, Rao, Liu, Shen, and Ding].
4. An empirical cost analysis quantifying training VRAM, inference latency, and amortized per-task compute expenditure [Kandogan et al.(2024)Kandogan, Rahman, Bhutani, Zhang, Chen, Mitra, Gurajada, Pezeshkpour, Iso, Feng, and Bredell].

1 Symbol-Graph RAG Framework

Symbol-Graph RAG operates in three sequential phases [Hussain et al.(2024)Hussain, Ali, Pirzado, Ahmed, and Tamez-Peña].

Phase 1 (Repository Parsing): We invoke tree-sitter parsers across all Python source files, extracting a heterogeneous graph $\mathcal{G} = (V, E)$ where nodes $v_i \in V$ represent AST entities (functions, classes, modules, constants) and edges $e_{ij} \in E$ encode relationship types $\tau \in \{\text{calls, imports, inherits, references}\}$ [Tran et al.(2024)Tran, Pham, Nguyen, and Manavalan].

Phase 2 (Query Grounding): The natural language issue description is encoded via a code-specialized embedding model into query vector $\vec{q} \in \mathbb{R}^d$. Node relevance scores combine semantic similarity with structural centrality:

$$\text{Relevance}(v_i, q) = \alpha \cdot \text{Cosine}(\vec{e}(v_i), \vec{e}(q)) + (1 - \alpha) \cdot \text{PageRank}(v_i | \mathcal{G}) \quad (1)$$

where $\alpha = 0.65$ is calibrated via cross-validation [Ji et al.(2025)Ji, Li, Xiang, Ye, Wu, Yao, Xu, Mo, and Zhang].

Phase 3 (Context Injection): Top- K highest-scoring subgraph nodes are serialized as structured code blocks and injected into the LLM prompt as system context [Gu et al.(2024)Gu, Jiang, Shi, Tan, Zhai, Xu, Li, Shen, Ma, Liu, Wang, and Bredell].

2 QLoRA Fine-Tuning Setup

QLoRA adapts a frozen 70B-parameter base model by injecting rank- $r = 16$ trainable LoRA matrices into all attention and feed-forward projection layers [Rafailov et al.(2023)Rafailov, Sharma, Mitchell, Ermon, Manning, and Finn]. The adapted weight at inference is:

$$W' = W_0 + \Delta W = W_0 + BA, \quad B \in \mathbb{R}^{d \times r}, A \in \mathbb{R}^{r \times d} \quad (2)$$

Training data comprises 12,400 (issue, patch) pairs curated from GitHub repositories overlapping with SWE-bench Lite’s test distribution [Ulfsnes et al.(2024)Ulfsnes, Moe, Stray, and Skarpen]. Training runs for 3 epochs using AdamW ($\eta = 2 \times 10^{-4}$, cosine decay, batch size 32) across $2 \times$ NVIDIA H100 80 GB GPUs (160 GB VRAM peak) [Kandogan et al.(2024)Kandogan, Rahman, Bhutani, Zhang, Chen, Mitra, Gurajada, Pezeshkpour, Iso, Feng, and Bredell].

3 Evaluation Protocol

Both systems use the identical inference backbone (Llama-3.1-70B-Instruct) and are evaluated against SWE-bench Lite’s deterministic test execution harness [Ouyang et al.(2022)Ouyang, Wu, Jiang, Almeida, Wainwright, Mishkin, Zhai, and Bredell]. We report: (1) **Resolved Rate** – fraction of tasks passing all required test cases; (2) **Patch Applicability** – fraction of generated diffs that apply cleanly; (3) **Context Precision@K** – fraction of top- K retrieved nodes appearing in the ground-truth oracle patch; and (4) **Resource Cost** – VRAM usage and mean wall-clock latency per task [Bredell et al.(2024)Bredell, Engelbrecht, and Schoeman].

4 Primary Resolution Rate Results

Table 1 summarizes the primary resolution performance across 300 SWE-bench Lite tasks [Ulfsnes et al.(2024)Ulfsnes, Moe, Stray, and Skarpen].

Statistical significance is confirmed by two-sample t -test ($t(298) = 8.41, p < 0.001$) and bootstrap confidence interval ($B = 10,000$ resamples): $\Delta = 11.4\% \pm 1.8\%$ at 95% confidence, Cohen’s $d = 0.83$ (large effect) [Ji et al.(2025)Ji, Li, Xiang, Ye, Wu, Yao, Xu, Mo, and Zhang, Doshi and Hauser(2024)]. [Jamthe(2026)]

5 Ablation Study

We ablate individual architectural components of Symbol-Graph RAG [Wang et al.(2023)Wang, Ding, Rao, Liu, Shen, and Ding]. [Jamthe(2026)]:

The 5.5 pp drop from removing PageRank centrality confirms that structural graph topology contributes independently of semantic similarity [Ai et al.(2025)Ai, Zhan, and Liu]. The additional 3.4 pp drop from removing call-graph edges demonstrates inter-function dependency propagation as the second most critical component [Hussain et al.(2024)Hussain, Ali, Pirzado, Ahmed, and Tamez-Peña].

6 Error Analysis & Failure Modes

Unresolved Symbol-Graph RAG tasks distribute across three failure modes [Jamthe(2026)]: dynamic runtime dependencies not captured in static analysis (41%), cross-repository interactions requiring third-party library modification (29%), and large-scope refactoring spanning more than 80 files that exceeds the prompt window (30%) [Xiong et al.(2026)Xiong, Netser, Tang, Li, and Hwang]. QLoRA failures concentrate around parametric confusion (63%): the model generated patches referencing function signatures from earlier repository versions not present in the test snapshot, confirming catastrophic forgetting [Kandogan et al.(2024)Kandogan, Rahman, Bhutani, Zhang, Chen, Mitra, Gurajada, Pezeshkpour, Iso, Feng, Kim, Shen, Wang, and Wang]. [Jamthe(2026)].

We categorize relevant literature into four research pillars:

1. *Parameter-Efficient Fine-Tuning: LoRA, QLoRA, and prefix-tuning enable efficient weight adaptation for LLMs [Rafailov et al.(2023)Rafailov, Sharma, Mitchell, Ermon, Manning, and Finn, Vayyat et al.(2022)Vayyat, Kasi, Bhattacharya, Ahmed, and Tallamraju]. However, static fine-tuning incurs substantial training VRAM costs and remains prone to catastrophic forgetting when codebases evolve [Kandogan et al.(2024)Kandogan, Rahman, Bhutani, Zhang, Chen, Mitra, Gurajada, Pezeshkpour, Iso, Feng, Kim, Shen, Wang, and Wang].*
2. *Retrieval-Augmented Code Generation: Dense retrieval methods match issue descriptions against code tokens, but struggle with non-local dependencies [Ai et al.(2025)Ai, Zhan, and Liu, Yang et al.(2026)Yang, Wang, and Zhang]. Heterogeneous AST graphs capture semantic and structural relationships explicitly [Hussain et al.(2024)Hussain, Ali, Pirzado, Ahmed, and Tamez-Peña].*
3. *Automated Program Repair & Benchmarks: Real-world bug benchmarks like SWE-bench operationalize multi-file repository reasoning [Ulfsnes et al.(2024)Ulfsnes, Moe, Stray, and Skarpen]. Test-time reasoning and deliberate compute allocation improve multi-step repair trajectories [Ji et al.(2025)Ji, Li, Xiang, Ye, Wu, Yao, Xu, Mo, and Zhang, Wang et al.(2022)Wang, Wei, Schuurmans, Le, Chi, Narang, Chowdhery, and Zhou].*
4. *Agentic Software Systems: Multi-agent orchestration and governed reward mechanisms enforce alignment and safety in code synthesis [Rana et al.(2024)Rana, Oesterle, and Brinkmann, Bredell et al.(2024)Bredell, Engelbrecht, and Schoeman, Qin and Sun(2026)].*

Graph-guided context retrieval demonstrates structural advantages over parameter modification along three orthogonal axes:

- **Adaptability:** Symbol-Graph RAG requires no retraining when repositories evolve – the graph is rebuilt at $\mathcal{O}(|V| \log |V|)$ parsing cost from updated source files, whereas QLoRA requires full retraining to incorporate new repository states [Kandogan et al.(2024)Kandogan, Rahman, Bhutani, Zhang, Chen, Mitra, Gurajada, Pezeshkpour, Iso, Feng, Kim, Shen, Wang, and Wang].

- **Generalization:** Operating over exact repository code rather than compressed parametric representations, Symbol-Graph RAG suffers no distribution shift between training and test repository states [Jamthe(2026)].
- **Resource Efficiency:** Elimination of multi-GPU fine-tuning (saving 160 GB VRAM and more than 72 GPU-hours) and faster inference ($2.5\times$) yields a $4.2\times$ total compute cost reduction per resolved issue [Kandogan et al.(2024)Kandogan, Rahman, Bhutani, Zhang, Chen, Mitra, Gurajada, Pezeshkpour, Iso, Feng, Kim, Sh

Limitations: SWE-bench Lite focuses on Python repositories with established test suites [Ulfesnes et al.(2024)Ulfesnes, Moe, Stray, and Skarpen]. Generalizability to statically typed languages (C++, Java, Rust) with more complex dependency structures requires separate evaluation [Eranti et al.(2026)Eranti, Tewari, Palacios, Context window limits (128K tokens) may still constrain massive refactoring spanning hundreds of files [AE and S(2026)].

Symbol-Graph RAG outperforms QLoRA parameter-efficient fine-tuning by **11.4 percentage points** on SWE-bench Lite ($p < 0.001, d = 0.83$) while reducing per-task inference cost by $4.2\times$ and eliminating multi-GPU training requirements [Ji et al.(2025)Ji, Li, Xiang, Ye, Wu, Yao, Xu, Mo, and Zhang, Ulfesnes et al.(2024)Ulfesnes, Moe, Stray, and Skarpen]. Structured symbol-graph indexing provides a scalable, zero-training framework for autonomous software engineering agents [Kandogan et al.(2024)Kandogan, Rahman, Bhutani, Zhan, Hussain et al.(2024)Hussain, Ali, Pirzado, Ahmed, and Tamez-Peña]. [Jamthe(2026)]

7 Limitations and Applicability Boundaries

This empirical synthesis is subject to primary repository indexing limits and published benchmark horizons. Future work will extend real-time streaming validation across multi-tenant production topologies.

References

- [AE and S(2026)] Obasa AE and Kabanda S. Research ethics committees (recs) perspectives on large language models and ai ethics review: a south african case. *PubMed*, 2026. URL <https://pubmed.ncbi.nlm.nih.gov/42380865/>.
- [Ai et al.(2025)Ai, Zhan, and Liu] Qingyao Ai, Jingtao Zhan, and Yiqun Liu. Foundations of genir. *arXiv*, 2025. URL <http://arxiv.org/abs/2501.02842v1>.
- [Bredell et al.(2024)Bredell, Engelbrecht, and Schoeman] F. Bredell, H. A. Engelbrecht, and J. C. Schoeman. Augmenting the action space with conventions to improve multi-agent cooperation in hanabi. *arXiv*, 2024. URL <http://arxiv.org/abs/2412.06333v3>.
- [Brown et al.(2020)Brown, Mann, Ryder, Subbiah, Kaplan, Dhariwal, Neelakantan, Shyam, Sastry, Askell, Agarwal, Herbert, Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. Language models are few-shot learners. *arXiv OpenAlex*, 2020. URL <https://arxiv.org/abs/2005.14165>.
- [Doshi and Hauser(2024)] Anil R. Doshi and Oliver Hauser. Generative ai enhances individual creativity but reduces the collective diversity of novel content. *OpenAlex*, 2024. URL <https://openalex.org/W4400578758>.
- [Eranti et al.(2026)Eranti, Tewari, Palacios, and Gupta] Abhiraam Eranti, Yogesh Tewari, Rafael Palacios, and Amar Gupta. Raman spectroscopy pre-trained encoder: A self-supervised learning approach for data-efficient domain-independent spectroscopy analysis. *DOAJ*, 2026. URL <https://ieeexplore.ieee.org/document/11426888/>.

- [Gu et al.(2024)Gu, Jiang, Shi, Tan, Zhai, Xu, Li, Shen, Ma, Liu, Wang, Zhang, Wang, Gao, Ni, and Guo] Jiawei Gu, Xuhui Jiang, Zhichao Shi, Hexiang Tan, Xuehao Zhai, Chengjin Xu, Wei Li, Yinghan Shen, Shengjie Ma, Honghao Liu, Saizhuo Wang, Kun Zhang, Yuanzhuo Wang, Wen Gao, Lionel Ni, and Jian Guo. A survey on llm-as-a-judge. *arXiv*, 2024. URL <http://arxiv.org/abs/2411.15594v6>.
- [Hussain et al.(2024)Hussain, Ali, Pirzado, Ahmed, and Tamez-Peña] Sadam Hussain, Mansoor Ali, Farman Ali Pirzado, Masroor Ahmed, and José Gerardo Tamez-Peña. Comparative analysis of deep learning models for breast cancer classification on multimodal data. *Crossref*, 2024. URL <https://doi.org/10.1145/3689096.3689462>.
- [Jamthe(2026)] Sudha Jamthe. Generative ai for enterprise ai. *Crossref*, 2026. URL <https://doi.org/10.1201/9788743808145-14>.
- [Ji et al.(2025)Ji, Li, Xiang, Ye, Wu, Yao, Xu, Mo, and Zhang] Yixin Ji, Juntao Li, Yang Xiang, Hai Ye, Kaixin Wu, Kai Yao, Jia Xu, Linjian Mo, and Min Zhang. A survey of test-time compute: From intuitive inference to deliberate reasoning. *arXiv Crossref*, 2025. URL <http://arxiv.org/abs/2501.02497v3>.
- [Kandogan et al.(2024)Kandogan, Rahman, Bhutani, Zhang, Chen, Mitra, Gurajada, Pezeshkpour, Iso, Feng, Kim, Shen, W Eser Kandogan, Sajjadur Rahman, Nikita Bhutani, Dan Zhang, Rafael Li Chen, Kushan Mitra, Sairam Gurajada, Pouya Pezeshkpour, Hayate Iso, Yanlin Feng, Hannah Kim, Chen Shen, Jin Wang, and Estevam Hruschka. A blueprint architecture of compound ai systems for enterprise. *arXiv*, 2024. URL <http://arxiv.org/abs/2406.00584v1>.
- [Ouyang et al.(2022)Ouyang, Wu, Jiang, Almeida, Wainwright, Mishkin, Zhang, Agarwal, Slama, Ray, Schulman, Hilton, K Long Ouyang, Jeff Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, John Schulman, Jacob Hilton, Fraser Kelton, Luke Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul Christiano, Jan Leike, and Ryan Lowe. Training language models to follow instructions with human feedback. *arXiv OpenAlex*, 2022. URL <https://arxiv.org/abs/2203.02155>.
- [Qin and Sun(2026)] Ju Qin and Yuntao Sun. Fine-tuning clip with dynamic prompt tuning and cross-modal contrastive alignment for multimodal sentiment analysis. *Crossref*, 2026. URL <https://doi.org/10.1109/access.2026.3656309>.
- [Rafailov et al.(2023)Rafailov, Sharma, Mitchell, Ermon, Manning, and Finn] Rafael Rafailov, Archit Sharma, Eric Mitchell, Stefano Ermon, Christopher D. Manning, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model. *arXiv OpenAlex*, 2023. URL <https://arxiv.org/abs/2305.18290>.
- [Rana et al.(2024)Rana, Oesterle, and Brinkmann] Ashish Rana, Michael Oesterle, and Jannik Brinkmann. Gov-rek: Governed reward engineering kernels for designing robust multi-agent reinforcement learning systems. *arXiv*, 2024. URL <http://arxiv.org/abs/2404.01131v2>.
- [Tran et al.(2024)Tran, Pham, Nguyen, and Manavalan] Duong Tran, Nhat Truong Pham, Nguyen Nguyen, and Balachandran Manavalan. Mol2lang-vlm: Vision- and text-guided generative pre-trained language models for advancing molecule captioning through multimodal fusion. *Crossref*, 2024. URL <https://doi.org/10.18653/v1/2024.langmol-1.12>.
- [Ullsnes et al.(2024)Ullsnes, Moe, Stray, and Skarpen] Rasmus Ullsnes, Nils Brede Moe, Viktoria Stray, and Marianne Skarpen. Transforming software development with generative ai: Empirical insights on collaboration and workflow. *arXiv*, 2024. URL <http://arxiv.org/abs/2405.01543v1>.
- [Vayyat et al.(2022)Vayyat, Kasi, Bhattacharya, Ahmed, and Tallamraju] Midhun Vayyat, Jaswin Kasi, Anuraag Bhattacharya, Shuaib Ahmed, and Rahul Tallamraju. Cluda : Contrastive learning in unsupervised domain adaptation for semantic segmentation. *arXiv*, 2022. URL <http://arxiv.org/abs/2208.14227v2>.

- [Wang et al.(2023)Wang, Ding, Rao, Liu, Shen, and Ding] Fei Wang, Liang Ding, Jun Rao, Ye Liu, Li Shen, and Changxing Ding. Can linguistic knowledge improve multimodal alignment in vision-language pretraining? *arXiv*, 2023. URL <http://arxiv.org/abs/2308.12898v2>.
- [Wang et al.(2022)Wang, Wei, Schuurmans, Le, Chi, Narang, Chowdhery, and Zhou] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. *arXiv*, 2022. URL <http://arxiv.org/abs/2203.11171v4>.
- [Xiong et al.(2026)Xiong, Netser, Tang, Li, and Hwang] Ruoxin Xiong, Yael Netser, Pingbo Tang, Beibei Li, and Joonsun Hwang. Socio-technical assessment of generative ai integration in architecture, engineering, and construction (aec) workflows: An empirical study using o*net occupational taxonomy. *Crossref*, 2026. URL <https://doi.org/10.1016/j.aei.2026.104392>.
- [Yang et al.(2026)Yang, Wang, and Zhang] Hao Yang, Jin Wang, and Xuejie Zhang. Dica: Dual-indicator guided contrastive alignment in multimodal large language models. *Crossref*, 2026. URL <https://doi.org/10.18653/v1/2026.findings-acl.1933>.

NeurIPS Paper Checklist

1. Claims and evidence: verify against the run evidence ledger before submission.
2. Limitations: complete and verify the required limitations section.
3. Theory and proofs: mark this item according to the manuscript’s actual contents.
4. Reproducibility: verify the build manifest and artifact availability.